Arith $\rightarrow$ IMP Compiler Correctness in Isabelle

Max Carroll

24/02/2026

1 Loop Invariants

We wish to show that if a while loop preserves some invariant on the state, and if the original state satisfies this invariant, then the resulting state after looping also must satisfy the invariant.

Further, the while conditional must be false after exiting the loop. This fact is independent of the invariant and can be proved separately. But, as we cannot distinguish the start and final states using the invariant[1], knowing the conditional is false can specialise/simplify the invariant on the final state into a meaningful statement which distinguishes it from the start state. We will see examples of this in the coming sections.

A simple case analysis can prove the `WhileFalse` case, but fails in the `WhileTrue` where we can only show that the invariant is preserved unto the next (existential) state, not to `t`. Hence, we need induction on the derivation of `(WHILE b DO c, s)` $\Rightarrow$ `t`.

The most basic property to prove on the derivation would be:

$$P(\texttt{(c,s')} \Rightarrow \texttt{t'}) := \neg\texttt{bval b t'} \wedge \texttt{Inv t'}$$

Which can be proven for the `WhileTrue` case, but not for the important base case `WhileFalse`, which requires P to hold over all 'false' while loops:

$$\forall r.\ \neg\texttt{bval b r} \Longrightarrow P(\texttt{(WHILE b DO c)} \Rightarrow \texttt{r})$$

That is:

$$\forall r.\ \neg\texttt{bval b r} \Longrightarrow \neg\texttt{bval b r} \wedge \texttt{Inv r}$$

But `r` is arbitrary, not generally equal to our start state `s` for which the invariant holds: `Inv s`. We cannot make any progress. The induction hypothesis must be weakened to account for this, for example:

$$P(\texttt{(c,s')} \Rightarrow \texttt{t'}) := \texttt{Inv s'} \Longrightarrow \neg\texttt{bval b t'} \wedge \texttt{Inv t'}$$

For which we combine `Inv s` with $P(\texttt{(c,s)} \Rightarrow \texttt{t})$ to get $\neg\texttt{bval b t} \wedge \texttt{Inv t}$ as required. The base case (`WhileTrue`) is trivial requiring conclusion:

$$P(\texttt{(WHILE b DO c, r)} \Rightarrow \texttt{r}) = \texttt{Inv r} \rightarrow \neg\texttt{bval b r} \wedge \texttt{Inv r} = \neg\texttt{bval b r}$$

[1] They both satisfy the invariant.

The `WhileTrue` case requires the preservation assumption to satisfy this implication relating `Inv s'` $\Rightarrow$ `Inv t'` over a single loop step.

In Isabelle, simply applying induction to `(WHILE b DO c, s)` $\Rightarrow$ `t` w.r.t. all 3 assumptions at once creates an induction hypothesis which can solve the `WhileTrue` case in a self-contained manner. This works by essentially also including the assumption directly within the induction hypothesis.

In my code I provide the induction tactic with the minimal assumptions so that the simpler hypothesis as above is chosen, then I explicitly provide the preservation assumption to auto for use in solving each case, but it also works encoding the preservation assumption into the induction hypothesis.

2 Defining the Minus Operator

There is trickiness ensuring that the definition calculates the correct values for negative values in the variables. In particular, intuitive solutions that involve counting *down* from `V 1` implicitly assume we are subtracting a non-negative number, subtracting a negative number will count up! Therefore, it is necessary to do a conditional on the sign of `V 1` (or on `V 0 + V 1` or similar).

There are many ways to solve this but I start from `V 0` and count down (or up if `V 1 < 0`) until `V 0 = V 1 + V 2`:

```
2 ::= V 0;;
(WHILE Not (eq (V 0) (Plus (V 1) (V 2))) DO
    (IF Less (V 1) (N 0)
     THEN 2 ::= Plus (V 2) (N 1)
     ELSE 2 ::= Plus (V 2) (N (−1))))
```

Where we define `eq a b` $\equiv$ `Not (And (Less a b) (Less b a))`.

This solution has the advantage of requiring only one while loop and only assigning variable 2, simplifying the proofs.

We can prove this using loop invariants. In particular from our boolean conditional `b` $\equiv$ `Not (eq (V 0) (Plus (V 1) (V 2)))`, we get almost our required result immediately (after simplification within the Isabelle type universe for integers & booleans after applying `bval`):

$$\neg\texttt{bval b t} = \neg\neg(\texttt{t 0 = t 1 + t 2}) = (\texttt{t 2 = t 0 - t 1})$$

Therefore the only invariant we need is to show that variables 0 and 1 are left untouched:

$$\texttt{Inv s'} := \texttt{s' 0 = s 0} \wedge \texttt{s' 1 = s 1}$$

Which is again trivial to prove as neither address is ever assigned in the loop body nor in the 'initialisation' line which sets `2 := V 0`.

However, this lemma does not actually prove the program *correct* in the total sense. At no point do we establish that the loop terminates. All the heavy lifting in the proof is done by knowing that the loop condition is false after termination, almost directly establishing the result; the loop invariant is

practically free. This means any loop body that doesn't modify `V 0` and `V 1` is perfectly valid, including many blatantly wrong ones. This is unsurprising as we can prove anything from the non-terminating loop. In fact it is much easier to find non-terminating examples that look reasonable, but fail to terminate on edge cases.[2]

A more 'reliable' solution could re-implement the while condition to decrement/increment `V 1` monotonically to zero while decrement/incrementing `V 2` accordingly starting from `V 2 = V 0` as before. This would require a more interesting loop invariant:

$$\texttt{Inv s'} := \texttt{s' 2 - s' 1 = s 0 - s 1}$$

Which is satisfied in initial state `s' = r` as `r 2 = s 0, r 1 = s 1` and in final state `s' = t` combined with `bval b t` $\equiv$ `t 1 = 0` gives `t 2 = s 0 - s 1` as required.

Of course this still does not prove termination, but in this case it is more obvious that `V 1` is monotonically[3] increasing/decreasing to zero. You could equip a size-order to the preservation lemma showing $|\texttt{Inv s'}| > |\texttt{Inv t'}| > 0$ for every `(c, s')` $\Rightarrow$ `t'` and `bval b t` $\wedge$ `Inv t` $\implies$ $|\texttt{Inv t}| = 0$ by taking the absolute value of `s' 1`. But this requires a fundamental change to the `While_inv` lemma.

Equally a size order can be assigned to my original solution showing that $|\texttt{s' 1 + s' 2 - s' 0}|$ is monotonically decreasing and equals 0 after termination. But this is more complex than simply taking the cardinality of `s' 1` as above.

I have not formalised any of this size-order termination in Isabelle as it seems out of scope (after all I proved the given lemma, and can informally confirm termination), but it is worth mentioning.

3 Arithmetic: `aexp2`

Evaluations of a arithmetic expression `a` in `aexp2` agree with each other when applied to states which agree for the free variables of `a`. This follows by the definition of free variables and by trivial induction on the arithmetic expression `a`.

Induction on the structure of the definition of `avars2` is exactly equivalent to the structure of `aexp2`. Another valid induction scheme is on the structure of `aval2` for either state `s` or `t`, but this adds unnecessary complexity.

4 Compiler for `aexp2` to `com`

I implement the compiler by reuse of the original `minus_com` from section 2. Then multiplication can be implemented by defining a command `mult_pos_com`

[2] As I found out by having Isabelle crash many times trying to run the command...

[3] In the strict sense.

which calculates $\mathtt{a} \times \mathtt{b}$ for non-negative `a`, which can be done by loops counting up to `a` via a temporary register, and adding `b` to the result each time starting from 0.

Then `minus_com 0 a`[4] can be used to define a unary minus command `unary_minus_com`. Multiplication `mult_com` is defined via `unary_minus_com` and `mult_pos_com` by negating the first argument followed by the result if required.

I structure these definitions in some sense like a assembly instruction with the following calling convention:

- Registers 0, 1 used for arguments and are *preserved* over calls.
- Register 2 is used (overwritten) for the result.
- (Temporary) Register 3 is corruptible in commands `unary_minus_com`, `mult_pos_com`, and `mult_com`.
- (Temporary) Register 4 is corruptible in command `mult_com`.

That is, at most registers 2, 3, or 4 could be corrupted by a command. So we would expect only arithmetic expressions referring to non-corruptible registers to compile correctly.

Finally, the full compiler stores its intermediate results for each compound operation in the register `res` and `res + 1`, which therefore corrupts the register `res`, but also recursively may corrupt a stack of d registers above `res` for the max depth d of expression e. And to perform operations must overwrite the argument registers 0 and 1.

Hence, for each recursive result to be correctly stored in `res` it must not be one machine registers above:[5] i.e. `res > 4`.

Further, compiling an expression which itself refers to any of the corruptible registers will not be correct in general. This issue is somewhat unfortunate. But is easily sidestepped by defining a mapping F from 'virtual names' v in the arithmetic expression to 'machine names' m in the compiled expression with a *retraction* R to return back again, for example:

$$k = \mathtt{res} + \mathrm{depth}(\mathtt{e})$$

$$F(v) := r \mapsto v(r+k) \qquad R(m) := \begin{cases} r \mapsto 0 & \text{if } r < k \\ r \mapsto m(r-k) & \text{if } r \geq k \end{cases}$$

Ideally the two 'state' functions for virtual and machine registers would be given different types so that they are not confused with each other.

However, since we are allowed to adapt the assumptions, I can simply require the expression `e` to not reference corruptible registers:[6]

$$(\{0,1,2,3,4\} \cup \{r \in \mathbb{N} \mid \mathtt{res} \leq r \leq \mathtt{res\ +\ depth(e)}\}) \cap (\mathtt{avars2\ e}) = \emptyset$$

[4] Where this notation means setting start state `s 0 = 0` and `s 1 = a`

[5] Including registers 0 and 1 which are no longer preserved.

[6] Written in implication form in the code. This seems to help the auto prover.

Leaving the selection / translation of suitable register naming schemes to the higher level.

In the next part, I will *overestimate* the region the compiler corrupts by including all `r > res` to avoid dealing with expression sizes. You can still compile any program if given freedom to pick sufficiently large `res`.[7]

5 Proving Compiler Correctness

Now onto the proofs. First we must prove that each command definition satisfies the calling convention, and agrees with the denotational semantics using `While_inv`; for start and end states `s`, `t`:

- `t 2 = s 1 <op> s 2` for corresponding operation `<op>`.
- Non-corruptible registers `r` are preserved: `t r = s r`. This includes registers 0 and 1. I refer to this as *frame preservation*[8] where the 'frame' is a set of preserved registers.

`minus_com` Partial Correctness

We could reuse the result from section 2 so that we only need to prove that `minus_com` preserves all registers except register 2, but in reality all we need to do is to strengthen the invariant to full frame preservation. The previous invariant equates to frame preservation just on registers 0 and 1, we can just extend this frame to all $\texttt{r} \neq 2$, the proof is no more complex. Note: I copy the proof again to keep my original solution to section 2 intact.

`mult_pos_com` Partial Correctness

For this we only require partial correctness when the first multiple (in register 0) is *nonnnegative* ($\texttt{s 0} \geq 0$), as specified in section 4.

Proof is again similar to above but with a much more interesting invariant:

$$\texttt{Inv s'} := \texttt{s' 3} \leq \texttt{s 0} \wedge \texttt{s' 2 = s' 3 * s 1}$$

Alongside the usual frame preservation part stating registers except 2 and 3 are unchanged (as specified). Also, $\texttt{s' 0} = \texttt{s 0}$ and $\texttt{s' 1} = \texttt{s 1}$ are used to prove the loop invariant parts of the start and preservation lemmas. Both also fall under the frame preservation part of the invariant so are omitted above, but would be additionally required if I was not proving the register calling convention in conjunction.

From this we get in a terminal state `t` that `t 2 = t 3 * s 1` and $\texttt{t 3} \leq \texttt{s 0}$. When combined the negated boolean conditional ($\texttt{t 3} \geq \texttt{t 0}$) and one instance

[7] Specifically, `res > 4 + avars e`, and using a suitable register naming translation.

[8] In some sense it can be seen as similar to the frame rule in Seperation logic, although under a very different formalism.

of preservation `t 0 = s 0` and antisymmetry of $\leq$ we get `t 3 = s 0`, hence `t 2 = s 0 * s 1` as required.

Isabelle can automate all the proofs as before. But, invariant preservation uses *more assumptions* than before; we need to prove that `s' 3` $+ 1 \leq$ `s 0` in order to ensure `t' 3` $\leq$ `s 0` over loop states `s'` to `t'`. This follows from the truthity of the **boolean conditional** stating: `s' 3` $<$ `t' 0` (= `s 0`). We also require a simple algebra lemma for $(x + 1) * y = x * y + y$ which is in `algebra_simps`.

Further, proving the start lemma requires `s 3` $\leq$ `s 0`, following from positivity assumption of `s 0` and that `s 3` is initialised to 0.

`unary_minus_com` Partial Correctness

Proven automatically from the correctness of `minus_com`. As it is just a simple chain of assignments.

`mult_com` Partial Correctness

Also automatic from correctness of `unary_minus_com` and `mult_pos_com`. Some trivial algebra is required to show correctness of $x * y = -((-x) * y)$.

Partial Correctness of aexp2

We will prove partial correctness under the weakened assumptions that:

- `res` is not a machine register (0 to 4)
- The expression does not reference any potentially corruptible registers. That is, none of: `r < 5` nor `r` $\geq$ `res`.

And we will prove a strengthened conclusion:

- Partial Correctness: If the compiled program terminates it produces a result agreeing with the denotation.
- (Full) Frame Preservation: All non-corrupting variables (`4 < r < res`) are preserved by the compiled program. By assumption, this includes all free variables of the expression.

Proof: Proceed by induction on the definition of `com_of_aexp2`. Importantly, the start state `s`, final state `t`, and preserved registers `r` must be kept *arbitrary* to allow use of the induction hypothesis on the differing intermediate states involved in each recursive calls, and ensuring the first recursive call stored in `res` is not corrupted by the second `com_of_aexp2 (res + 1)` call.[9]

The second part of interest is that the denotation acts equivalently on intermediate state as it does in the start state (`aval2 a' s' = aval2 a' s`).

[9] This is the a key reason the compiler works, and is the reason that `res` is incremented in recursive calls

True because free variables of the expression are non-corruptable by assumption which, by frame preservation, allows the use of the congruence rule from section 3.

Note: For some reason `auto` really struggles with this (infinite looping and introducing weird universal quantifiers). So my code has a full proof layout, of which the structure is almost identical between the 3 non-automatic cases.

6 Termination

We can build derivations of a while loop by induction on the natural numbers corresponding to the number of *remaining loops* in the derivation (calculated from the state `s`). I call this the *cardinality* of the given loop state notated `|s|`.[10]

This means the base case is just proving the loop whose condition is false rule for all states `|s| = 0`, which permits the construction of a single-rule derivation via the `WhileFalse` rule. While the inductive case requires you to:

1. Prove the loop condition holds (as `|s| > 0`, i.e. iterations remain).
2. Run the loop once from a state `|s|` = `n + 1` and prove the resulting state has `n` iterations remaining: `|s'| = n`.
3. By induction construct the derivation for the remaining `n` iterations from state `s'` (requiring the induction hypothesis to be arbitrary on states).
4. Use the `WhileTrue` rule to construct a derivation from `s` (with $n + 1$ remaining loops `|s| = n + 1`).

To demonstrate, I have a fully written out example theorem in my code proving that the simple loop which just decrements register 0 to zero always terminates. In which case the cardinality function is simple: `|s| = max(s 0, 0)`. i.e. the register's value is exactly the number of remaining loops (unless register is negative, when the loop terminates immediately).

Of course, any loop that does not terminate cannot prove the inductive step for a non-terminating state.

`minus_com` Terminates

As it turns out I do have to prove termination of my answer to section 2 in the end. I can use the argument as discussed in section 2 that the difference between the current result and the target result counts exactly the number of remaining iterations. That is, induct on the absolute value `|s 2 - (s 0 - s 1)|`.

This chain is more complex than the others, register 2 starts at the value of the first register and then decreases until it becomes the difference (or increases if

[10] This is not super general for proving termination of more complex languages, especially any with non-determinism, but is all we need for this section.

the subtractor is negative). If this were not true, we would not have termination. So the states considered must preserve a variant, that `s 2` always lies *between* `s 0` and the result `s 0 - s 1`.

Because the sign of the subtractor `s 1` can be negative, formalising and proving this invariant requires many cases (in code represented by if statements). Luckily as this is mostly arithmetic, Isabelle could still automate all these extra calculations once a suitable invariant was defined.

`minus_pos_com` Terminates

The multiplication loop is simpler to prove, with register 3 counting directly up towards the value in register 0, so this difference is a suitable natural number to induct upon. The invariant guaranteeing termination must therefore requiring register 3 to be always less or equal to register 0. Since register 3 is set to 0 at the start, this means that the command as a whole terminates if register 0 is positive, as is consistent with my specification (`mult_pos_com` performs valid multiplication for non-negative first arguments).

`unary_com` and `mult_com` Terminate

These are just chains of terminating commands. For which I use two simple but useful dual rules stating how termination proofs can be chained:

- A command terminating to a known terminal start `s'` may **prepend** a command that terminates startng from `s'`. Note that `s'` must be known to be *consistent* with the start state of the latter command; we can't just rely on the basic termination property (terminates to *some existential* state).
- If a command terminates for *all* start states then you may `append` it to a command which terminates from some state. Symmetrically to the above, this universal quantifier is necessary as we do not necessarily know in which state the first command terminates, so we must prove termination from all starting states of the second.

Plus, an if statement terminates if the to-be executed branch terminates.

Proving multiplication terminates requires the partial correctness results for `unary_minus_com` to guarantee that `mult_pos_com` is never given a negative argument in register 0. My concise proof uses associativity of sequences to inject this guarantee bridging the halves `mult_pos_com` command, where termination of each half can be automated.

Full Compiler Termination

Simply proceed by induction on the definition of the compiler. The compiler just chains commands and assignments, always terminate, including for states invalid in the partial correctness sense.

Surprisingly auto can't solve the cases immediately. Metis can according to sledgehammer but on my laptop this was crashing, so I have directly inserted the sledgehammer-generated proofs here.

Word Count: 2576